



**SRI RAMAKRISHNA
ENGINEERING COLLEGE**

[Educational Service : SNR Sons Charitable Trust]

Vattamalaipalayam, N.G.G.O. Colony Post,

Coimbatore – 641022.

 www.srec.ac.in

 +917530089996



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACTIVE LEARNING METHODOLOGY REPORT

**LPC2148 BASED MULTI-SENSOR FAULT DETECTION SYSTEM
USING INTERRUPTS, TIMERS AND SAFE SHUTDOWN**

BHAVANA S (71812401027)
CHANDRA M (71812401028)
CHRISTINA DAMARIS E (71812401029)
CHRISWIN JANUSZ A (71812401030)
DHAKSHITHA S (71812401031)

THIRD YEAR B.E. CSE – V SEM

Academic Year 2026-2027

**20EC252 & MICROCONTROLLER
AND PROGRAMMING**

COURSE COORDINATOR

Mrs. Amuthasurabi M, AP/CSE



SRI RAMAKRISHNA ENGINEERING COLLEGE

[Educational Service : SNR Sons Charitable Trust]

Vattamalaipalayam, N.G.G.O. Colony Post,

Coimbatore – 641022.

www.srec.ac.in

+917530089996



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

20EC252 & MICROCONTROLLER AND PROGRAMMING

INDEX

S. No.	Name & Roll No.	Tool Explanation (2)	Implementation (5)	Presentation (3)	Total (10)
1.	BHAVANA S (71812401027)				
2.	CHANDRA M (71812401028)				
3.	CHRISTINA DAMARIS E (71812401029)				
4.	CHRISWIN JANUSZ A (71812401030)				
5.	DHAKSHITHA S (71812401031)				

SIGNATURE OF COURSE COORDINATOR

Mrs. Amuthasurabi M, AP/CSE



SRI RAMAKRISHNA ENGINEERING COLLEGE

[Educational Service : SNR Sons Charitable Trust]

Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.

 www.srec.ac.in

 +917530089996



CONTENTS

S. No.	TITLE	PAGE NUMBER
1.	INTRODUCTION	1
2.	WORKFLOW DESCRIPTION	2
3.	IMPLEMENTATION STEPS	4
4.	CODE IMPLEMENTATION	6
5.	OUTPUT	14
6.	DIAGRAMMATIC REPRESENTATION	19
7.	CONCLUSION	24

INTRODUCTION

Industrial machines, electrical panels, boilers and processing equipment can develop dangerous conditions such as overheating, gas leakage, excessive current or fire. If these conditions are not detected and acted upon quickly, they can damage equipment, stop production and endanger people. A microcontroller-based fault detection system continuously watches the plant through sensors, warns the operator when values start to drift, and shuts the equipment down safely when a limit is crossed. The proposed project, “LPC2148 Based Multi-Sensor Fault Detection System Using Interrupts, Timers and Safe Shutdown”, implements such a protection system using the LPC2148 ARM7 microcontroller.

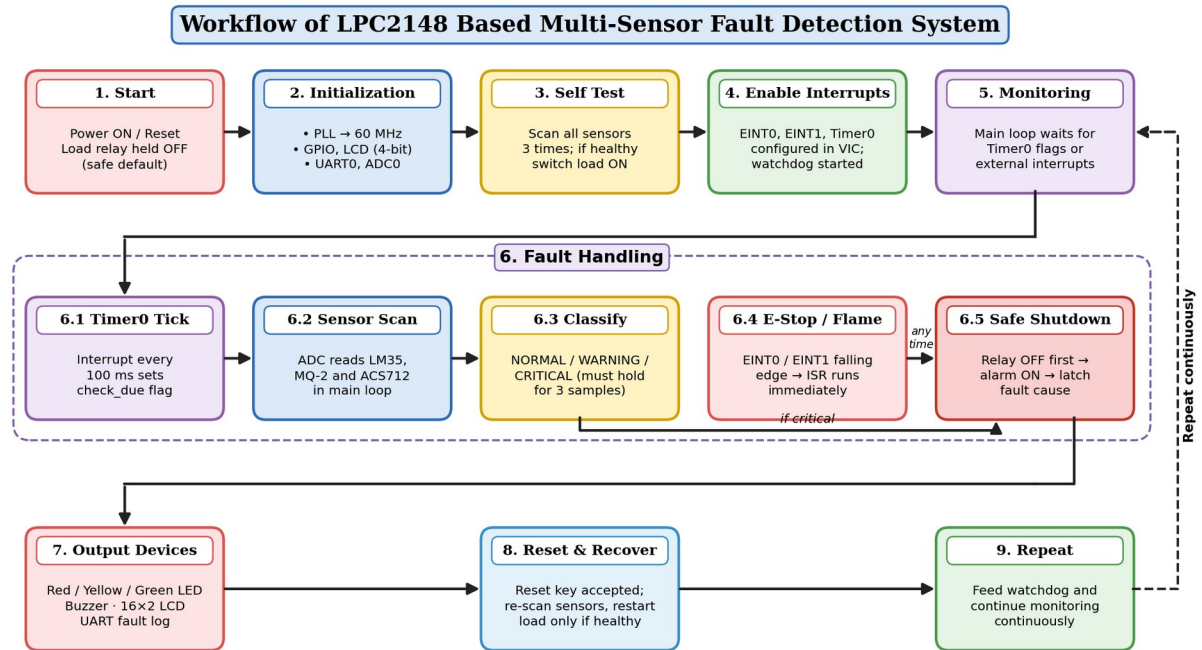
The LPC2148 is a 32-bit ARM7TDMI-S microcontroller with 512 KB of flash memory, 40 KB of RAM and a maximum operating frequency of 60 MHz using its on-chip PLL. It integrates all the peripherals needed for this application: a 10-bit ADC for reading analog sensors, 32-bit timers for periodic scheduling, external interrupt inputs (EINT0–EINT3), a Vectored Interrupt Controller (VIC), a watchdog timer, UART serial ports and general-purpose I/O pins.

The system monitors five inputs. Three analog sensors measure quantities that change gradually: an **LM35** temperature sensor, an **MQ-2** gas/smoke sensor and an **ACS712** current sensor, connected to ADC channels AD0.1–AD0.3. Two digital inputs represent events that need an instant response: a **flame sensor** on P0.3/EINT1 and an **emergency-stop push button** on P0.16/EINT0. The outputs are a load relay that powers the protected equipment, a buzzer, red/yellow/green status LEDs, a 16×2 LCD in 4-bit mode and a UART fault log.

The key design question is how to structure the software. The proposed design uses **external interrupts** for emergency events so that they are handled within microseconds, **Timer0 interrupts every 100 ms** to schedule periodic sensor checking without blocking delays, and a dedicated **safe shutdown routine** that always switches the load off first, raises the alarm and latches the fault until a manual reset. A three-state machine (NORMAL, WARNING, SHUTDOWN), a three-sample confirmation filter against false alarms and the on-chip watchdog timer complete the design.

The firmware is developed in Embedded C using Keil µVision. The generated HEX file is loaded into Proteus to verify the operation of the sensors, interrupts, relay, alarms and LCD before implementing the system on hardware. The system can be applied to motor protection, industrial safety panels, server-room monitoring, kitchen and boiler safety, and laboratory equipment protection.

WORKFLOW DESCRIPTION



DESCRIPTION:

The workflow of the proposed fault detection system consists of initialization, self-testing, continuous monitoring, fault classification, safe shutdown, alarm output and recovery. The system begins at the Start block when power is applied or the reset button is pressed. The very first action is to hold the load relay OFF, so that the protected equipment never runs before the system has confirmed that conditions are safe.

In the Initialization stage, the PLL is configured to raise the system clock from the 12 MHz crystal to 60 MHz. The GPIO pins for the relay, buzzer and LEDs are configured as outputs and the reset key as an input. The 16×2 LCD is initialized in 4-bit mode, UART0 is set to 9600 baud for the fault log, and pins P0.28–P0.30 are switched to their ADC function.

Next, the Self Test stage scans all sensors three times. Only if every reading is below its critical limit is the load relay switched ON and the green LED lit. The system then enables its interrupts: EINT0 and EINT1 are configured for falling-edge detection, Timer0 is started with a 100 ms period, all three sources are registered in the VIC, and the watchdog timer is started. The microcontroller now enters the Monitoring stage, where the main loop waits for flags set by interrupts.

SOFTWARE STRUCTURE – INTERRUPTS:

Events that cannot wait are handled through external interrupts. Pressing the emergency-stop button produces a falling edge on P0.16/EINT0, and a flame detection pulls P0.3/EINT1 low. The VIC immediately transfers control to EINT0_ISR or EINT1_ISR. Emergency stop is given VIC slot 0 (highest priority), flame slot 1 and Timer0 slot 2. Each of these ISRs simply calls safe_shutdown(), clears its EXTINT flag and writes VICVectAddr = 0. Because they do very little work, they finish in microseconds and the response does not depend on what the main loop was doing.

SOFTWARE STRUCTURE – TIMER-BASED PERIODIC CHECKING:

Temperature, gas and current change slowly and reading the ADC takes time, so these sensors are checked periodically rather than inside an interrupt. Timer0 is programmed with a prescaler of 59 (1 MHz count) and a match value of 100000, so it interrupts every 100 ms. TIMER0_ISR only sets the flag `check_due`. The main loop sees the flag and calls `scan_sensors()`, which reads the three ADC channels and compares each value with its warning and critical limits. A reading must stay above the critical limit for three consecutive scans (300 ms) before it is treated as a real fault; this filters out noise spikes. If any value is above its warning limit, the system enters the WARNING state, and the Timer0 ISR blinks the yellow LED and beeps the buzzer.

SOFTWARE STRUCTURE – SAFE SYSTEM SHUTDOWN:

Every critical fault, whether detected by an interrupt or by the periodic scan, goes through one function, `safe_shutdown()`. It first de-energizes the load relay to remove the hazard, then switches on the red LED and continuous buzzer, latches the first fault cause in `fault_code` and sets `log_due` so that the main loop sends the fault record over UART. The relay is wired so that the load runs only while the coil is energized; therefore a power failure or a CPU reset also switches the load off. The shutdown is latched: the system restarts only when the reset key is pressed and a fresh sensor scan confirms that all values are back to normal. The watchdog timer, fed once in every main-loop pass, resets the microcontroller if the software ever hangs, which also releases the relay.

In the Output stage, the LCD shows the live temperature and gas readings together with NORMAL, WARNING or the fault name, and the LEDs and buzzer give a visual and audible indication. Finally, the system feeds the watchdog and returns to monitoring. The process repeats continuously for as long as the system is powered.

Thus, the complete workflow can be summarized as:

Power ON/Reset → Load OFF → Initialization → Self Test → Enable Interrupts & Watchdog → Monitor → Timer0 Tick → Sensor Scan → Classify (Normal / Warning / Critical) → E-Stop/Flame Interrupt → Safe Shutdown → Alarm & LCD & UART Log → Reset & Re-check → Repeat Continuously.

IMPLEMENTATION STEPS

The implementation of the LPC2148-based fault detection system is carried out using Keil μ Vision for program development and compilation, and Proteus for simulation. The complete implementation involves creating the project, selecting the microcontroller, adding the source program, configuring the target, compiling, generating the HEX file and finally simulating the circuit.

Step 1: Open Keil μ Vision

Open the installed Keil μ Vision 5 (MDK-ARM) development environment. The MDK-ARM version is used because the LPC2148 is an ARM7-based microcontroller. Select Project → New μ Vision Project, give a suitable name such as LPC2148_Fault_Detection and choose a separate folder for the project files.

Step 2: Create a New Project

After the project is saved, Keil opens the Select Device for Target window. Selecting the correct device is important because Keil uses this information to configure the registers, memory areas, startup files and compiler settings.

Step 3: Select LPC2148 Device

Search for LPC2148 and select it under NXP → LPC2000 → LPC2100 Series, then click OK. When Keil asks whether to add the startup file (Startup.s) to the project, select Yes; it sets up the stacks and the interrupt vector table used by the VIC.

Step 4: Add the Source File

Right-click Source Group 1, choose Add New Item to Group, select C File (.c) and save it as main.c. This file holds the complete program for ADC sensor reading, Timer0 and external interrupt handling, LCD and UART output, and safe shutdown.

Step 5: Write the Program

Write the program inside main.c, including the LPC214x.h header to access the LPC2148 registers. The program configures the PLL, GPIO directions, LCD, UART0 and ADC; defines EINT0_ISR, EINT1_ISR and TIMER0_ISR using the __irq keyword; and implements scan_sensors(), safe_shutdown(), the state machine and the watchdog routines. Save the file using Ctrl + S.

Step 6: Configure Target Options

Open Options for Target and enter the crystal frequency as 12.000 MHz in the Target tab. The firmware configures the PLL itself, so the CPU and peripheral clock run at 60 MHz.

Step 7: Build the Project

Compile the project using Build Target or F7. The Build Output window shows the compilation status, program size, errors and warnings. A successful build should report 0 errors; any warnings should be checked.

Step 8: Generate the HEX File

Open Options for Target → Output, enable Create HEX File and click OK. Rebuild the project so that Keil generates the .hex file required by the Proteus LPC2148 model.

Step 9: Design the Circuit and Load the HEX File in Proteus

Place the LPC2148, a 12 MHz crystal with 33 pF capacitors, an LM016L LCD with a contrast potentiometer, an LM35 temperature sensor, two potentiometers to simulate the MQ-2 and ACS712 outputs, push buttons with pull-up resistors for emergency stop (P0.16) and reset (P0.4), a logic toggle for the flame sensor (P0.3), red/yellow/green LEDs, a buzzer, a 2N2222 transistor with a 1N4007 diode driving a relay with a lamp as the load, and a Virtual Terminal on P0.0/P0.1. Double-click the LPC2148, select the HEX file under Program File and set the clock frequency to 12 MHz.

Step 10: Run the Simulation

Click the Run button. After the self test, the load lamp and green LED switch ON and the LCD shows STATUS: NORMAL.

Step 11: Observe the Output

Increase the LM35 temperature or the potentiometer values to create warning and critical conditions, press the emergency-stop button, toggle the flame input and press the reset key. Verify that the relay, LEDs, buzzer, LCD and Virtual Terminal respond as listed in the test cases.

Step 12: Project Completion

Successful compilation and correct simulated behaviour for all test cases verify the operation of the fault detection system. The project now contains the LPC2148 configuration, the embedded C program, the HEX file and the Proteus simulation circuit.

CODE IMPLEMENTATION

The software of the proposed fault detection system is developed in Embedded C using Keil μ Vision for the LPC2148 ARM7 microcontroller. The LPC214x.h header file is included to access the special function registers. The program is organized into a driver layer (PLL, LCD, UART, ADC, timer and interrupt set-up), an interrupt layer (EINT0, EINT1 and Timer0 ISRs), an application layer (sensor scan and state machine in the main loop) and a safety layer (safe_shutdown() and the watchdog).

The emergency stop and flame sensor are connected to P0.16/EINT0 and P0.3/EINT1. The LM35, MQ-2 and ACS712 are connected to AD0.1–AD0.3 on P0.28–P0.30. The load relay is on P0.15, the buzzer on P0.10 and the LEDs on P0.11–P0.13. The LCD uses P1.16–P1.21 in 4-bit mode, and UART0 uses P0.0/P0.1.

PIN CONFIGURATION:

S. No.	Function	LPC2148 Pin
1.	UART0 Tx/D / Rx/D	P0.0 / P0.1
2.	Flame Sensor / EINT1	P0.3
3.	Fault Reset Key	P0.4
4.	Buzzer	P0.10
5.	Red LED (Critical)	P0.11
6.	Yellow LED (Warning)	P0.12
7.	Green LED (Normal)	P0.13
8.	Load Relay	P0.15
9.	Emergency Stop / EINT0	P0.16
10.	LM35 Temperature / AD0.1	P0.28
11.	MQ-2 Gas / AD0.2	P0.29
12.	ACS712 Current / AD0.3	P0.30
13.	LCD RS	P1.16
14.	LCD EN	P1.17
15.	LCD D4 – D7	P1.18 – P1.21

THRESHOLD VALUES:

Sensor	Warning Level	Critical Level (3 samples)
LM35 Temperature	$\geq 50\text{ }^{\circ}\text{C}$	$\geq 70\text{ }^{\circ}\text{C}$
MQ-2 Gas	≥ 600 ADC counts	≥ 800 ADC counts
ACS712 Current	≥ 700 ADC counts	≥ 850 ADC counts
Flame Sensor	–	Immediate (EINT1)
Emergency Stop	–	Immediate (EINT0)

Temperature is calculated as $ADC \times 330 / 1023$ because the ADC reference is 3.3 V and the LM35 gives 10 mV/°C. Gas and current limits are raw counts to be calibrated for the actual sensor module.

PROGRAM:

```
#include <LPC214x.h>

/* ----- Pin definitions ----- */
#define BUZZER (1 << 10) /* P0.10 */
#define LED_RED (1 << 11) /* P0.11 - Critical fault */
#define LED_YEL (1 << 12) /* P0.12 - Warning */
#define LED_GRN (1 << 13) /* P0.13 - Normal */
#define LOAD_RELAY (1 << 15) /* P0.15 - Main load supply */
#define RESET_KEY (1 << 4) /* P0.4 - Fault reset key */

#define LCD_RS (1 << 16) /* P1.16 */
#define LCD_EN (1 << 17) /* P1.17 */
#define LCD_D4 (1 << 18) /* P1.18 */
#define LCD_D5 (1 << 19) /* P1.19 */
#define LCD_D6 (1 << 20) /* P1.20 */
#define LCD_D7 (1 << 21) /* P1.21 */
#define LCD_DATA (LCD_D4 | LCD_D5 | LCD_D6 | LCD_D7)

/* ----- Thresholds ----- */
#define TEMP_WARN 50 /* deg C */
#define TEMP_CRIT 70
#define GAS_WARN 600 /* raw ADC count */
#define GAS_CRIT 800
#define CURR_WARN 700
#define CURR_CRIT 850
#define CONFIRM_CNT 3 /* consecutive samples to confirm */

/* ----- System states ----- */
#define ST_NORMAL 0
#define ST_WARNING 1
#define ST_SHUTDOWN 2

/* ----- Fault codes ----- */
#define F_NONE 0
#define F_TEMP 1
#define F_GAS 2
#define F_CURRENT 3
#define F_FLAME 4
#define F_ESTOP 5

volatile unsigned char check_due = 0; /* set by Timer0 ISR */
volatile unsigned char sys_state = ST_NORMAL;
volatile unsigned char fault_code = F_NONE;
volatile unsigned char log_due = 0;
volatile unsigned int tick_100ms = 0;
volatile unsigned char disp_due = 0; /* LCD refresh request */

unsigned int temp_c, gas_val, curr_val;
unsigned char temp_cnt = 0, gas_cnt = 0, curr_cnt = 0;

const char *fault_text[] = { "NONE", "OVER TEMP", "GAS LEAK",
                             "OVER CURRENT", "FLAME", "E-STOP" };;
```

```

/* ----- Clock: 12 MHz crystal -> CCLK = PCLK = 60 MHz ----- */
void pll_init(void)
{
    PLL0CFG = 0x24; /* M = 5, P = 2 */
    PLL0CON = 0x01; /* enable PLL */
    PLL0FEED = 0xAA; PLL0FEED = 0x55;
    while(!(PLL0STAT & (1 << 10))); /* wait for lock */
    PLL0CON = 0x03; /* connect PLL */
    PLL0FEED = 0xAA; PLL0FEED = 0x55;
    VPBDIV = 0x01; /* PCLK = CCLK */
}

void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    for(i = 0; i < ms; i++)
        for(j = 0; j < 6000; j++);
}

/* ----- LCD (4-bit, Port 1) ----- */
void lcd_nibble(unsigned char n)
{
    IO1CLR = LCD_DATA;
    if(n & 0x01) IO1SET = LCD_D4;
    if(n & 0x02) IO1SET = LCD_D5;
    if(n & 0x04) IO1SET = LCD_D6;
    if(n & 0x08) IO1SET = LCD_D7;
    IO1SET = LCD_EN; delay_ms(1); IO1CLR = LCD_EN;
}

void lcd_cmd(unsigned char c)
{
    IO1CLR = LCD_RS;
    lcd_nibble(c >> 4); lcd_nibble(c & 0x0F);
    delay_ms(2);
}

void lcd_char(unsigned char d)
{
    IO1SET = LCD_RS;
    lcd_nibble(d >> 4); lcd_nibble(d & 0x0F);
    delay_ms(1);
}

void lcd_string(const char *s) { while(*s) lcd_char(*s++); }

void lcd_num(unsigned int n)
{
    char buf[6]; int i = 0;
    if(n == 0) { lcd_char('0'); return; }
    while(n) { buf[i++] = (n % 10) + '0'; n /= 10; }
    while(i) lcd_char(buf[--i]);
}

void lcd_init(void)
{
    IO1DIR |= LCD_RS | LCD_EN | LCD_DATA;
    delay_ms(20);
    lcd_nibble(0x03); delay_ms(5);
}

```

```

    lcd_nibble(0x03); delay_ms(1);
    lcd_nibble(0x03); lcd_nibble(0x02);
    lcd_cmd(0x28); lcd_cmd(0x0C); lcd_cmd(0x06); lcd_cmd(0x01);
}

/* ----- UART0: 9600 baud @ PCLK 60 MHz ----- */
void uart0_init(void)
{
    PINSEL0 |= 0x00000005; /* P0.0 TxD0, P0.1 RxD0 */
    U0LCR = 0x83; /* 8N1, DLAB = 1 */
    U0DLL = 0x87; /* 391 = 0x0187 */
    U0DLM = 0x01;
    U0LCR = 0x03; /* DLAB = 0 */
}

void uart0_char(char c) { while(!(U0LSR & 0x20)); U0THR = c; }
void uart0_string(const char *s) { while(*s) uart0_char(*s++); }

void uart0_num(unsigned int n)
{
    char buf[6]; int i = 0;
    if(n == 0) { uart0_char('0'); return; }
    while(n) { buf[i++] = (n % 10) + '0'; n /= 10; }
    while(i) uart0_char(buf[--i]);
}

/* ----- ADC0: channels 1, 2, 3 ----- */
void adc_init(void)
{
    PINSEL1 &= ~(3 << 24 | 3 << 26 | 3 << 28);
    PINSEL1 |= ((1 << 24) | (1 << 26) | (1 << 28)); /* P0.28-P0.30 = AD0.1-AD0.3 */
}

unsigned int adc_read(unsigned char ch)
{
    unsigned int val;
    AD0CR = (1 << ch) | (13 << 8) | (1 << 21) | (1 << 24); /* 4.28 MHz, start */
    while(!(AD0GDR & (1UL << 31))); /* wait DONE */
    val = (AD0GDR >> 6) & 0x3FF;
    AD0CR &= ~(7 << 24); /* stop */
    return val;
}

/* ----- Safe shutdown (callable from ISR or main) ----- */
void safe_shutdown(unsigned char code)
{
    IO0CLR = LOAD_RELAY; /* 1. cut power to load first */
    IO0CLR = LED_GRN | LED_YEL;
    IO0SET = LED_RED | BUZZER; /* 2. raise alarm */
    if(sys_state != ST_SHUTDOWN)
        fault_code = code; /* 3. latch first fault cause */
    sys_state = ST_SHUTDOWN;
    log_due = 1; /* 4. main loop logs on UART */
}

/* ----- Timer0 ISR: 100 ms system tick ----- */
__irq void TIMER0_ISR(void)
{
    tick_100ms++;
}

```

```

    check_due = 1; /* request periodic sensor scan */
    if((tick_100ms % 5) == 0) disp_due = 1; /* LCD every 500 ms */
    if(sys_state == ST_WARNING) /* blink warning LED/buzzer */
    {
        if(tick_100ms & 0x04) IO0SET = LED_YEL | BUZZER;
        else IO0CLR = LED_YEL | BUZZER;
    }
    T0IR = 0x01; /* clear MR0 flag */
    VICVectAddr = 0;
}

/* ----- EINT0 ISR: Emergency stop ----- */
__irq void EINT0_ISR(void)
{
    safe_shutdown(F_ESTOP);
    EXTINT = (1 << 0);
    VICVectAddr = 0;
}

/* ----- EINT1 ISR: Flame sensor ----- */
__irq void EINT1_ISR(void)
{
    safe_shutdown(F_FLAME);
    EXTINT = (1 << 1);
    VICVectAddr = 0;
}

/* ----- Timer0: 100 ms periodic interrupt ----- */
void timer0_init(void)
{
    T0TCR = 0x02; /* reset timer */
    T0PR = 59; /* 60 MHz / 60 = 1 MHz */
    T0MR0 = 100000; /* 100 ms */
    T0MCR = 0x03; /* interrupt + reset on MR0 */
    T0TCR = 0x01; /* start */
}

/* ----- External interrupts + VIC ----- */
void interrupt_init(void)
{
    PINSEL1 &= ~(3 << 0); PINSEL1 |= (1 << 0); /* P0.16 = EINT0 */
    PINSEL0 |= (3 << 6); /* P0.3 = EINT1 */

    EXTMODE |= (1 << 0) | (1 << 1); /* edge sensitive */
    EXTPOLAR &= ~((1 << 0) | (1 << 1)); /* falling edge */
    EXTINT = (1 << 0) | (1 << 1); /* clear flags */

    VICIntSelect = 0x00000000; /* all IRQ */

    VICVectAddr0 = (unsigned)EINT0_ISR; VICVectCntl0 = 0x20 | 14; /* highest */
    VICVectAddr1 = (unsigned)EINT1_ISR; VICVectCntl1 = 0x20 | 15;
    VICVectAddr2 = (unsigned)TIMER0_ISR; VICVectCntl2 = 0x20 | 4;

    VICIntEnable = (1 << 14) | (1 << 15) | (1 << 4);
}

/* ----- Watchdog: resets CPU if software hangs (~2 s) ----- */
void wdt_init(void)
{

```

```

    WDTC = 30000000; /* 2 s x (60 MHz / 4) */
    WDMOD = 0x03; /* enable + reset on timeout */
    WDFEED = 0xAA; WDFEED = 0x55;
}

void wdt_feed(void)
{
    unsigned int saved = VICIntEnable;
    VICIntEnClr = 0xFFFFFFFF; /* no IRQ between feed bytes */
    WDFEED = 0xAA; WDFEED = 0x55;
    VICIntEnable = saved;
}

/* ----- Confirm a reading over N consecutive samples ----- */
unsigned char confirm(unsigned int val, unsigned int limit, unsigned char *cnt)
{
    if(val >= limit) { if(*cnt < 255) (*cnt)++; }
    else *cnt = 0;
    return (*cnt >= CONFIRM_CNT);
}

/* ----- Periodic sensor scan (called from main) ----- */
void scan_sensors(void)
{
    unsigned char warn = 0;

    temp_c = (adc_read(1) * 330) / 1023; /* LM35: 10 mV/degC */
    gas_val = adc_read(2);
    curr_val = adc_read(3);

    if(confirm(temp_c, TEMP_CRIT, &temp_cnt)) { safe_shutdown(F_TEMP);
    return; }
    if(confirm(gas_val, GAS_CRIT, &gas_cnt)) { safe_shutdown(F_GAS); return; }
    if(confirm(curr_val, CURR_CRIT, &curr_cnt)) { safe_shutdown(F_CURRENT);
    return; }

    if(temp_c >= TEMP_WARN || gas_val >= GAS_WARN || curr_val >=
    CURR_WARN)
        warn = 1;

    if(sys_state == ST_SHUTDOWN) return; /* ISR may have tripped meanwhile */
    if(warn) {
        sys_state = ST_WARNING;
        IO0CLR = LED_GRN | LED_RED;
    } else {
        sys_state = ST_NORMAL;
        IO0CLR = LED_YEL | LED_RED | BUZZER;
        IO0SET = LED_GRN;
    }
}

void show_status(void)
{
    lcd_cmd(0x80);
    lcd_string("T:"); lcd_num(temp_c); lcd_string("C G:");
    lcd_num(gas_val); lcd_string(" ");
    lcd_cmd(0xC0);
    if(sys_state == ST_NORMAL) lcd_string("STATUS: NORMAL ");
    else if(sys_state == ST_WARNING) lcd_string("STATUS: WARNING ");
}

```

```

    else { lcd_string("FLT:"); lcd_string(fault_text[fault_code]); lcd_string(" "); }
}

void log_fault(void)
{
    uart0_string("\r\n*** FAULT: ");
    uart0_string(fault_text[fault_code]);
    uart0_string(" | T="); uart0_num(temp_c);
    uart0_string("C GAS="); uart0_num(gas_val);
    uart0_string(" I="); uart0_num(curr_val);
    uart0_string(" | LOAD OFF ***\r\n");
}

int main(void)
{
    unsigned char i;

    pll_init();

    IO0DIR |= BUZZER | LED_RED | LED_YEL | LED_GRN | LOAD_RELAY;
    IO0DIR &= ~RESET_KEY;
    IO0CLR = BUZZER | LED_RED | LED_YEL | LOAD_RELAY; /* safe default
    */

    lcd_init();
    uart0_init();
    adc_init();

    lcd_cmd(0x80); lcd_string("FAULT DETECTION ");
    lcd_cmd(0xC0); lcd_string("SELF TEST... ");
    delay_ms(1000);

    for(i = 0; i < CONFIRM_CNT; i++) { /* power-on self test */
        scan_sensors();
        delay_ms(100);
    }
    if(sys_state != ST_SHUTDOWN) {
        IO0SET = LOAD_RELAY | LED_GRN; /* start load only if healthy */
        uart0_string("\r\nSystem started: all sensors normal\r\n");
    }

    interrupt_init();
    timer0_init();
    wdt_init();

    while(1)
    {
        if(check_due && sys_state != ST_SHUTDOWN) {
            check_due = 0;
            scan_sensors();
        }
        if(log_due) { log_due = 0; log_fault(); }

        if(sys_state == ST_SHUTDOWN && !(IO0PIN & RESET_KEY)) {
            delay_ms(50); /* debounce reset key */
            temp_cnt = gas_cnt = curr_cnt = 0;
            sys_state = ST_NORMAL;
            scan_sensors(); /* re-check before restart */
            if(sys_state != ST_SHUTDOWN) {

```

```
        fault_code = F_NONE;
        IO0CLR = BUZZER | LED_RED;
        IO0SET = LOAD_RELAY | LED_GRN;
        uart0_string("Fault cleared, load restarted\r\n");
    }
}

if(disp_due) { disp_due = 0; show_status(); }
wdt_feed();
}
```

OUTPUTS

CREATING A NEW μ VISION PROJECT:

[Paste screenshot here]
Keil – Create New μ Vision Project dialog (LPC2148_Fault_Detection)

SELECTING LPC2148 DEVICE:

[Paste screenshot here]
Keil – Select Device for Target window with LPC2148 highlighted

ADDING MAIN.C SOURCE FILE:

[Paste screenshot here]
Keil – Add New Item to Group dialog with main.c

EMBEDDED C PROGRAM IN KEIL:

[Paste screenshot here]
Keil – editor showing the fault detection program

TARGET CONFIGURATION:

[Paste screenshot here]
Keil – Options for Target, Xtal = 12.0 MHz

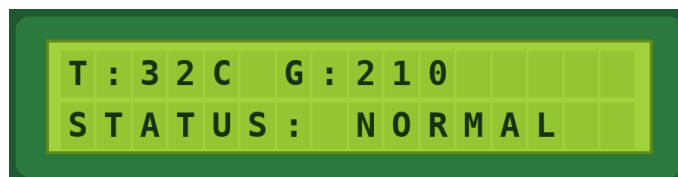
SUCCESSFUL BUILD OUTPUT:

[Paste screenshot here]
Keil – Build Output showing 0 Error(s)

HEX FILE GENERATION:

[Paste screenshot here]
Keil – Output tab with Create HEX File ticked

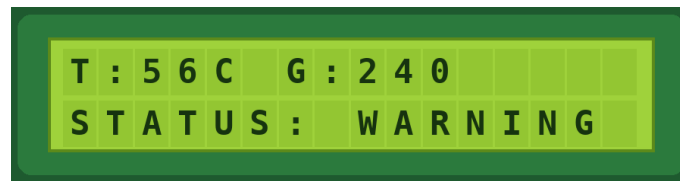
NORMAL CONDITION:



Expected LCD: T:32C G:210 / STATUS: NORMAL

[Paste screenshot here]
Proteus – load lamp ON, green LED ON

WARNING CONDITION (TEMPERATURE 56 °C):



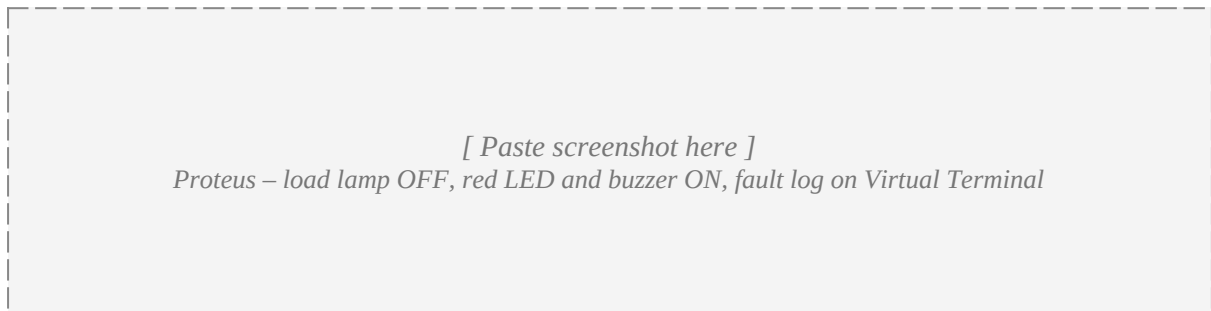
Expected LCD: STATUS: WARNING



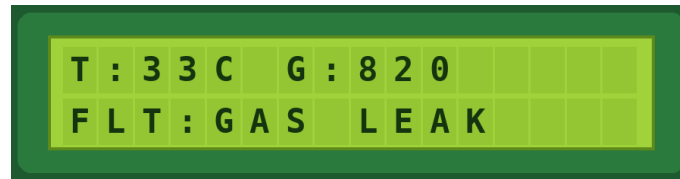
CRITICAL FAULT – OVER TEMPERATURE (74 °C):



Expected LCD: FLT:OVER TEMP



CRITICAL FAULT – GAS LEAK:



Expected LCD: FLT:GAS LEAK

[Paste screenshot here]

Proteus – load OFF after gas potentiometer exceeds 800 counts

EMERGENCY STOP ACTIVATED:

[Paste screenshot here]

Proteus – E-stop pressed, load OFF immediately, LCD shows FLT:E-STOP

EXPECTED VIRTUAL TERMINAL OUTPUT:

```
System started: all sensors normal
*** FAULT: OVER TEMP | T=74C GAS=250 I=310 | LOAD OFF ***
Fault cleared, load restarted
```

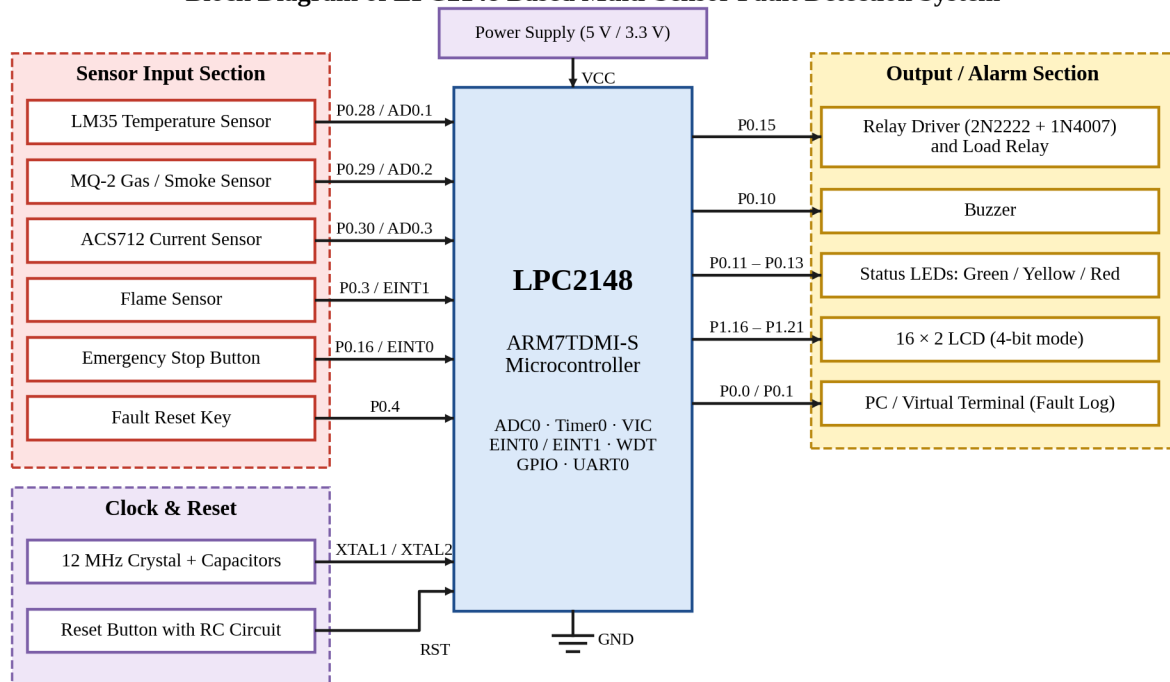
TEST CASES:

No.	Input Condition	Expected Output
1	All sensors normal (T = 32 °C, gas = 210)	Load ON, green LED, "STATUS: NORMAL"
2	Temperature raised to 56 °C	Load ON, yellow LED blinks, intermittent beep, "STATUS: WARNING"
3	Temperature 74 °C for more than 300 ms	Load OFF, red LED, continuous buzzer, "FLT:OVER TEMP", UART log
4	Gas value above 800 counts	Load OFF, "FLT:GAS LEAK"
5	Current value above 850 counts	Load OFF, "FLT:OVER CURRENT"
6	Emergency stop pressed	Immediate load OFF via EINT0, "FLT:E-STOP"
7	Flame input pulled low	Immediate load OFF via EINT1, "FLT:FLAME"
8	Reset pressed while fault still present	System stays in SHUTDOWN
9	Reset pressed after values return to normal	Fault cleared, load restarted

DIAGRAMMATIC REPRESENTATION

BLOCK DIAGRAM:

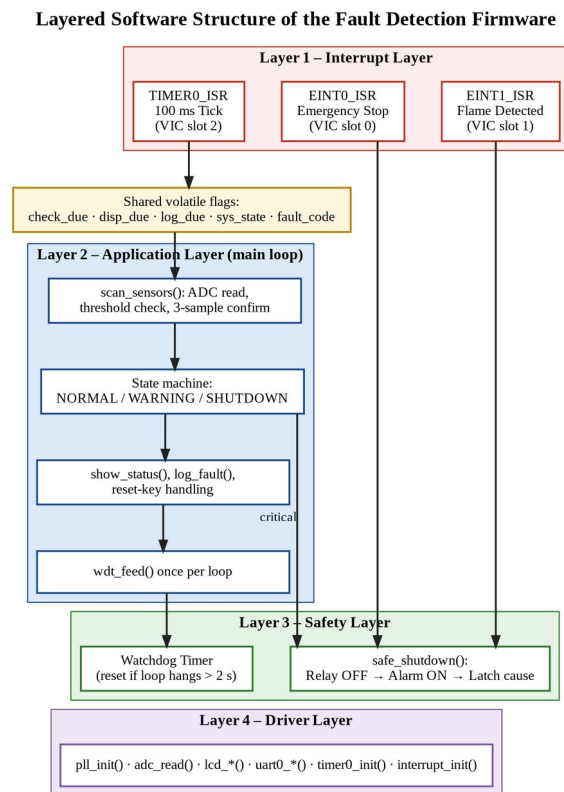
Block Diagram of LPC2148 Based Multi-Sensor Fault Detection System



The block diagram represents the complete hardware architecture of the fault detection system, with the LPC2148 ARM7TDMI-S microcontroller as the central control unit. The sensor input section contains the LM35, MQ-2 and ACS712 connected to ADC channels AD0.1–AD0.3, and the flame sensor and emergency-stop button connected to the external interrupt pins EINT1 and EINT0. The 5 V sensor modules are connected through resistor dividers so that the ADC input never exceeds 3.3 V.

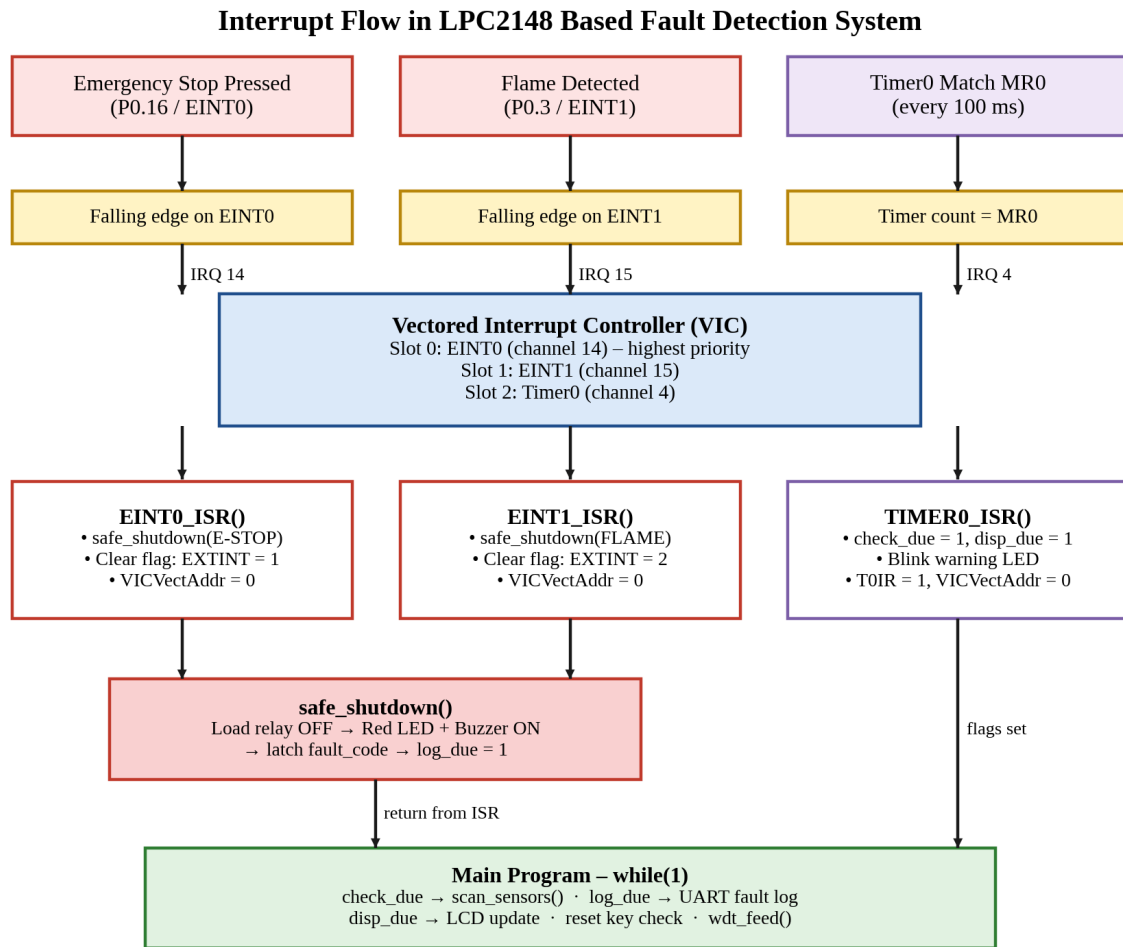
The output section contains the relay driver (2N2222 transistor with a 1N4007 flyback diode) controlling the load relay through P0.15, the buzzer, the three status LEDs, the 16×2 LCD in 4-bit mode on P1.16–P1.21 and the UART link to a PC. The clock circuit (12 MHz crystal with PLL) and the reset circuit provide timing and initialization. Signals flow from the sensors into the controller, which processes them and drives the alarm and switching outputs.

SOFTWARE STRUCTURE:



The firmware is divided into four layers. The interrupt layer contains short ISRs that only react or set flags. The application layer is the main super-loop, which performs the sensor scan, runs the state machine, updates the LCD and UART and feeds the watchdog. The safety layer contains `safe_shutdown()` and the watchdog timer, and the driver layer contains the low-level routines for the ADC, LCD, UART, timers and interrupts. Shared variables between ISRs and the main loop are declared volatile.

INTERRUPT FLOW:

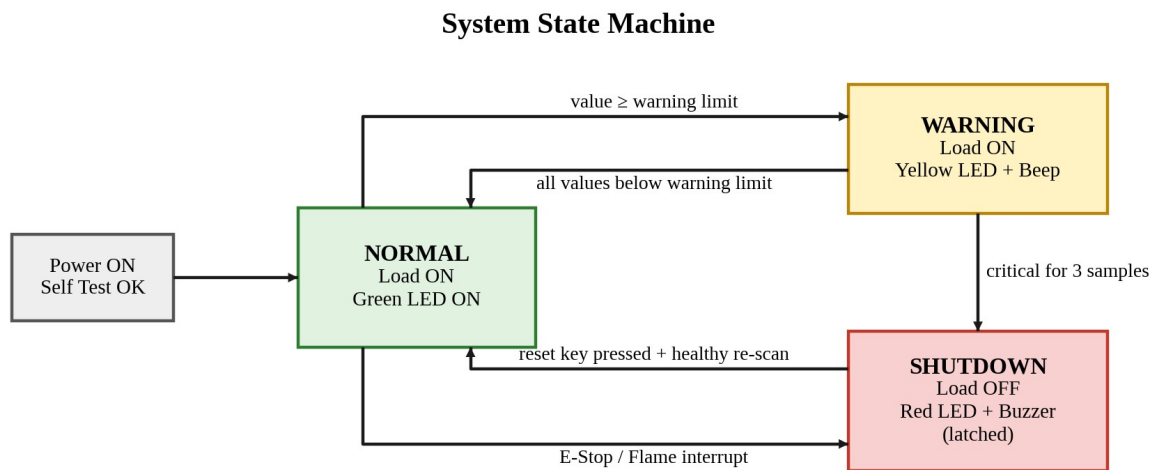


When the emergency-stop button is pressed or a flame is detected, a falling edge is generated on EINT0 or EINT1. The VIC identifies the source and transfers control to the corresponding ISR, which calls `safe_shutdown()`, clears the interrupt flag and returns. Timer0 generates an interrupt every 100 ms; its ISR sets the `check_due` and `disp_due` flags that the main program uses to schedule sensor scanning and display updates. After every ISR, control returns to the main program.

INTERRUPT SEQUENCE:

Event (E-Stop / Flame / Timer0) → VIC (priority) → ISR → Safe Shutdown or Set Flag → Clear Flag → Return to Main Program → Scan / Log / Display → Repeat

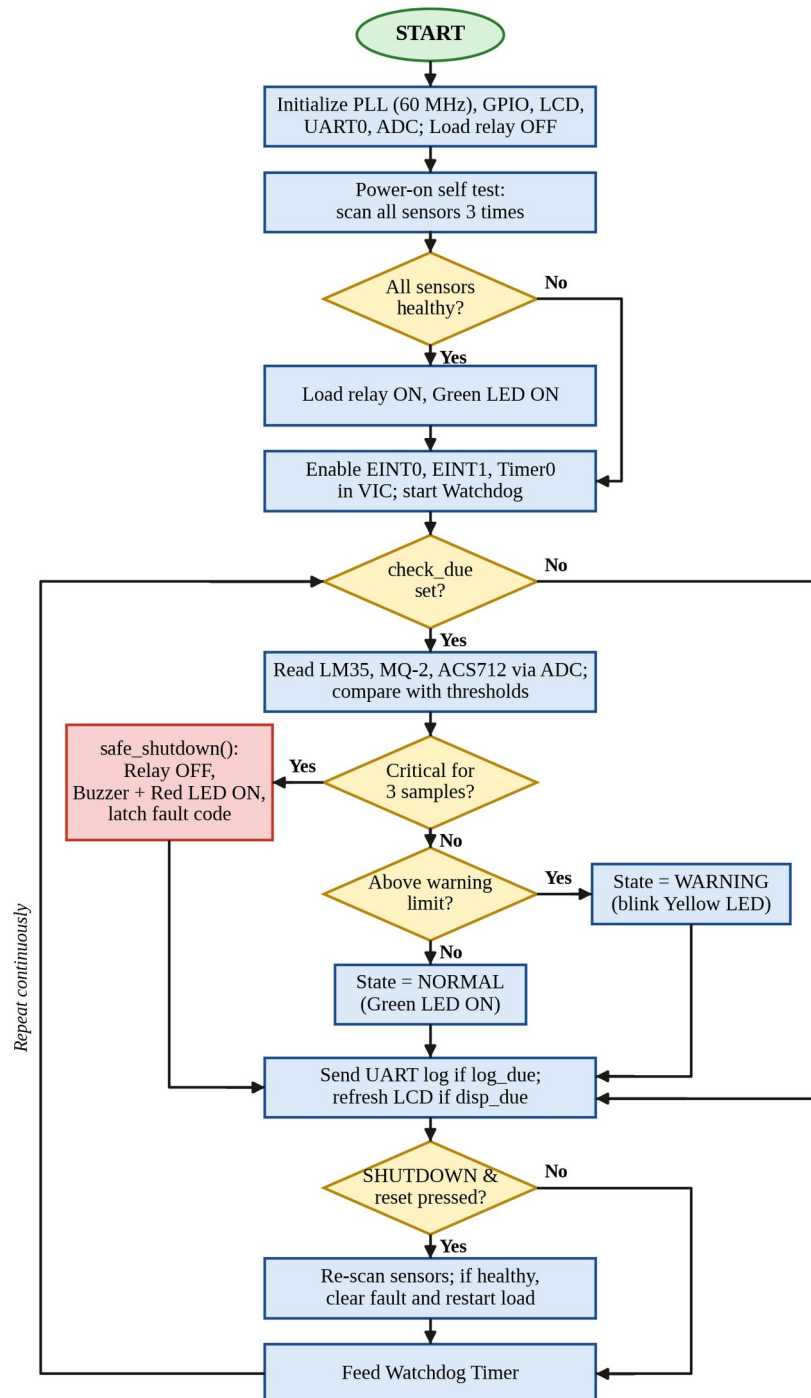
STATE MACHINE:



The system is always in one of three states. In **NORMAL** the load runs with the green LED on. In **WARNING** the load continues to run while the yellow LED blinks and the buzzer beeps. In **SHUTDOWN** the load is switched off with the red LED and continuous buzzer; this state is latched and is left only when the reset key is pressed and the re-scan confirms healthy conditions.

SYSTEM WORKFLOW:

Flowchart of LPC2148 Based Fault Detection System



CONCLUSION

The LPC2148-based multi-sensor fault detection system was designed to demonstrate how an ARM7 microcontroller can protect equipment by monitoring several sensors, raising alarms and shutting the system down safely when an abnormal condition occurs. The LPC2148 coordinates the temperature, gas, current and flame sensors, the emergency-stop input, the load relay, the alarm outputs, the LCD and the UART log.

The project answers the software-structure question through a clear division of responsibilities. External interrupts EINT0 and EINT1, given the highest VIC priorities, handle the emergency stop and flame detection instantly. Timer0 generates a 100 ms periodic interrupt that schedules sensor scanning, while the actual ADC reading and decision-making are done in the main loop so that the ISRs remain short. A three-sample confirmation filter prevents false alarms, and the NORMAL–WARNING–SHUTDOWN state machine separates early warnings from critical faults.

Safe shutdown is guaranteed through a single `safe_shutdown()` routine that switches the load off first, raises the alarm and latches the fault cause. Fail-safe relay wiring, a safe state at power-up, a power-on self test, a controlled manual reset with re-checking of all sensors, and the watchdog timer together ensure that the system always fails into a safe condition, even if the software hangs or power is lost.

The 16×2 LCD and the UART fault log give the operator clear information about the live sensor readings and the cause of any shutdown. The program was written and compiled in Keil μ Vision and converted into a HEX file for simulation in Proteus, allowing the complete behaviour to be verified before building the hardware.

The project provided practical experience in PLL clock configuration, ADC programming, timer and external interrupt handling through the VIC, GPIO interfacing, LCD and UART communication, and safety-oriented embedded software design. It can be extended by storing fault history in EEPROM, sending SMS alerts through a GSM module, adding more sensors such as vibration or voltage monitors, or connecting the system to an IoT dashboard for remote monitoring.

In conclusion, the proposed system successfully demonstrates how a microcontroller can detect abnormal conditions using interrupts and timer-based periodic checking, respond with alarms, and bring the system to a safe shutdown, forming a practical foundation for industrial safety and condition-monitoring applications.